

Quantum AI: From Inspiration to Enhancement - A Practical Journey

Physics-informed, physics-enhanced, physics-inspired

Dr. Pavel Sulimov Claude Lehmann

SDS2026 Workshop

07.06.2026

SDS2026

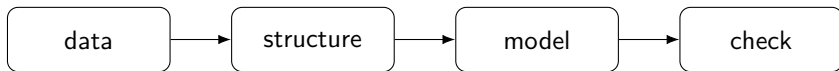
DATA. IMPACT. VALUE.



From inspiration to enhancement

The question stays the same throughout:

what structure can we add to a plain AI model?



The structure may come from the problem itself. We will add it step by step and check what it buys us.

Physics-informed learning

We start with a simple question: can a neural network learn a curve and respect a known rule?

Example rule:

$$u'(t) = -u(t), \quad u(0) = 1.$$

Lab file: 01_ode_physics_informed_networks.ipynb

Section: Model problem

The toy system

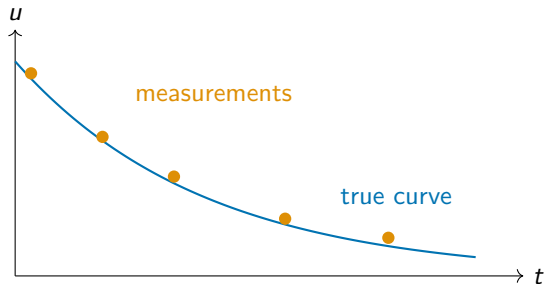
The solution is exponential decay:

$$u(t) = e^{-t}.$$

t	e^{-t}	interpretation
0	1.00	initial value
1	0.37	decay after one time unit
2	0.14	much smaller, but not zero

Data points alone

Suppose we only observe a few noisy points.



A flexible model can pass through points and still behave badly between them.

Add the rule to the loss

A usual data loss asks:

$$\hat{u}_\theta(t_i) \approx y_i.$$

The physics term asks:

$$\hat{u}'_\theta(t) + \hat{u}_\theta(t) \approx 0.$$

The model is still a neural network. The training signal is richer.

One loss, three terms

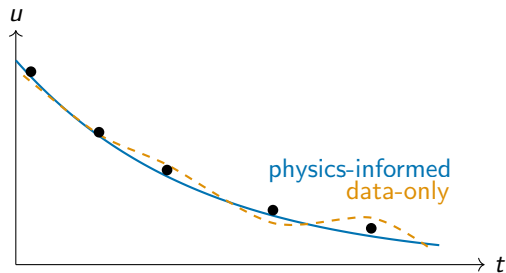
$$\mathcal{L}(\theta) = \text{data error} + \lambda \text{equation error} + \mu \text{initial error}.$$

For the toy ODE:

$$\text{equation error} = \frac{1}{K} \sum_{k=1}^K (\hat{u}'_{\theta}(\tau_k) + \hat{u}_{\theta}(\tau_k))^2.$$

The points τ_k are places where we check the equation.

Data-only versus physics-informed



This picture is the point. The equation discourages shapes that fit data but ignore the dynamics.

What can go wrong

The method is simple, but not automatic.

- Too much equation weight: the model may ignore useful data.
- Too little equation weight: the rule barely matters.
- Poor check points: the model can fail between them.

Lab file: 01_ode_physics_informed_networks.ipynb

Section: Fit both variants

Bitstrings as decisions

Many hard applied problems ask for a structured choice, not a smooth curve.

We write a candidate answer as bits:

$$z = (z_1, \dots, z_n), \quad z_i \in \{0, 1\}.$$

The important object is a cost function over candidates, not the name of a particular optimisation format.

Subset sum: the smallest useful example

Given weights $w = (3, 5, 6, 7, 9)$ and target $T = 15$, choose a subset:

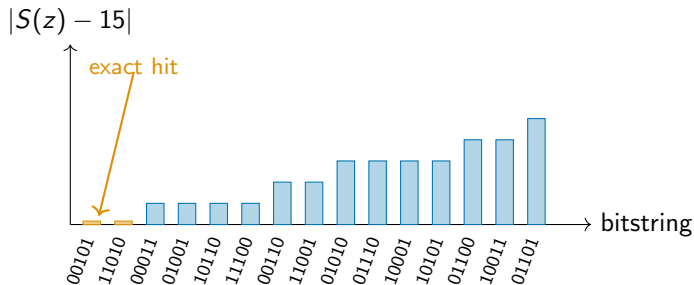
$$S(z) = \sum_i z_i w_i, \quad z_i \in \{0, 1\}.$$

item	A	B	C	D	E
weight	3	5	6	7	9

Example: $z = 10101$ selects A, C, E and gives $3 + 6 + 9 = 18$.

Subset sum as a landscape

Each bitstring is one candidate. The score is distance from the target.



Lab file: 02_hard_problems_bitstrings.ipynb

Section: Visualise subset-sum candidates

A bit is a modelling choice

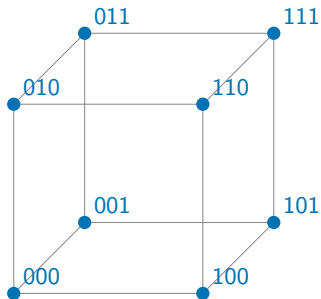
The same bit value can mean different things depending on the encoding:

- $z_i = 1$: item i is selected;
- $z_{i,t} = 1$: city i is visited at position t ;
- $z_j = 1$: Boolean variable x_j is true.

The encoding decides what the bitstring means and how validity is checked.

Search spaces grow geometrically

Three bits form the corners of a cube.



Exhaustive search over n bits has 2^n candidates.

TSP: a route is a constrained assignment

For cities A, B, C, D , a route is an ordering.

A binary encoding can use

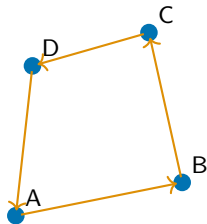
$$z_{i,t} = 1 \iff \text{city } i \text{ is visited at position } t.$$

Validity requires

$$\sum_i z_{i,t} = 1, \quad \sum_t z_{i,t} = 1.$$

The first constraint fills each time slot; the second visits each city once.

TSP: visual route score



candidate route: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

The score is the total route length. The constraints rule out strings that do not describe a tour.

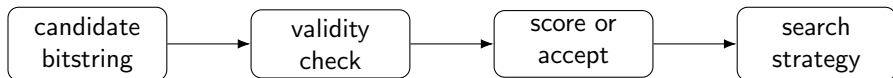
3SAT: a truth assignment is a bitstring

Variables x_1, x_2, x_3 become bits. Example formula:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3).$$

A candidate assignment such as $z = 101$ means $x_1 = \text{true}$, $x_2 = \text{false}$, $x_3 = \text{true}$.

Many problems, same outer shape



problem	candidate	check or score
subset sum	chosen items	distance from target
TSP	city order	route length + validity
3SAT	truth assignment	satisfied clauses

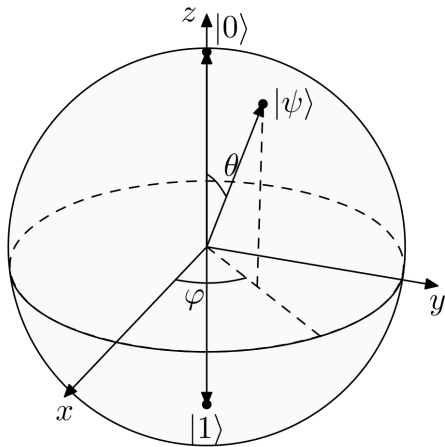
From candidate strings to sampled strings

Quantum circuits are interesting here because measurement produces classical bitstrings.

A useful circuit would not merely produce bits. It would change the sampling distribution so that good candidates appear more often.

The modelling work remains classical: define the encoding, evaluate candidates, and check whether the samples solve the intended problem.

Qubit & Bloch Sphere



A single qubit has basis states

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

A general pure state is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1.$$

Measurement

Measurement in the computational basis returns a classical bit:

$$\Pr(0) = |\alpha|^2, \quad \Pr(1) = |\beta|^2.$$

For n qubits,

$$|\psi\rangle = \sum_{z \in \{0,1\}^n} \alpha_z |z\rangle, \quad \Pr(z) = |\alpha_z|^2.$$

Repeated measurements sample $\{|\alpha_z|^2\}_z$.

One outcome per shot; the full distribution requires many shots.

Measurement is destructive: outcome z **collapses** the state to $|z\rangle$. All other amplitudes are **lost**. Unlike classical bits, quantum states cannot be read without being disturbed.

Quantum Gates

- **Unitary Operations:** All quantum gates are *unitary* matrices U , satisfying $U^\dagger U = I$. This ensures that quantum evolution is reversible.
- **State Transformation:** A gate U acts on a quantum state $|\psi\rangle$ as

$$|\psi'\rangle = U|\psi\rangle.$$

Multi-qubit gates act on subsets of qubits via tensor products with identities:

$$U_{\text{multi}} = I \otimes \cdots \otimes U \otimes \cdots \otimes I.$$

- **Reversibility:** Because U is unitary, every gate has an inverse:

$$U^{-1} = U^\dagger,$$

allowing “forward” and “backward” application of gates.

Single-Qubit Gates

X Gate (NOT)

Qiskit: `x(q)`



$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Y Gate

Qiskit: `y(q)`



$$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

Z Gate

Qiskit: `z(q)`



$$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Hadamard

Qiskit: `h(q)`

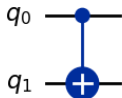


$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Two- and Multi-Qubit Gates

CX Gate (CNOT)

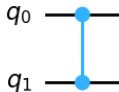
Qiskit: `cx(c1, t1)`



$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

CZ Gate

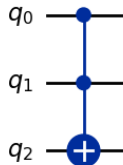
Qiskit: `cz(c1, t1)`



$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

Toffoli Gate (CCX)

Qiskit: `ccx(c1, c2, t1)`



$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Hadamard Gate: Superposition

The Hadamard gate creates an **equal superposition** of computational basis states:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Applying the Hadamard gate to the basis states $|0\rangle$ and $|1\rangle$ produces the **Plus state** $|+\rangle$ and **Minus state** $|-\rangle$.

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle$$

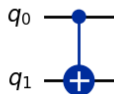
$$H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle$$

A single measurement gives one classical bit. The probability distribution becomes visible only through *repeated* measurements (**shots**).

CNOT Gate: Entanglement

The controlled-NOT (CNOT) gate flips the target qubit if the control qubit is $|1\rangle$.

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$



The gate acts on two qubits: the first is the control and the second is the target.

$$\text{CNOT} |00\rangle = |00\rangle$$

$$\text{CNOT} |10\rangle = |10\rangle$$

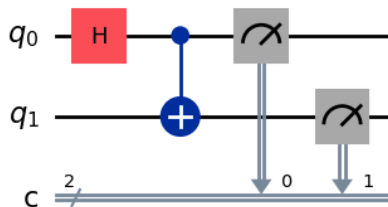
$$\text{CNOT} |01\rangle = |11\rangle$$

$$\text{CNOT} |11\rangle = |01\rangle$$

If the control qubit is $|0\rangle$, the target remains unchanged. If the control qubit is $|1\rangle$, the target qubit is flipped.

Bell State Circuit in Code

We now translate the Bell state from math into code.



$$|00\rangle \xrightarrow{H \otimes I} \frac{1}{\sqrt{2}} |00\rangle + \frac{1}{\sqrt{2}} |10\rangle \xrightarrow{\text{CNOT}_{0 \rightarrow 1}} \frac{1}{\sqrt{2}} |00\rangle + \frac{1}{\sqrt{2}} |11\rangle = |\Phi^+\rangle.$$

After this step, the system is in the **Bell state** $|\Phi^+\rangle$, which is fully entangled.

Expected measurement outcomes:

Approximately 50% for $|00\rangle$ and 50% for $|11\rangle$, with zero probability for $|01\rangle$ or $|10\rangle$.

Statevector vs Shot-Based Counts

Quantum circuits can be evaluated in two fundamentally different ways: through exact classical simulations (statevector), or repeated executions (shot-based).

Statevector simulators

- Return the **exact amplitudes** α_x
- No sampling noise
- Useful for debugging and theoretical analysis

Shot-based simulators and hardware

- Return **measurement counts** from repeated runs
- Finite-shot statistical uncertainty
- Hardware introduces gate and readout noise (or simulate comparable levels of noise)

Theory predicts probabilities; experiments estimate them through sampling.

What Changes on Real Hardware?

Moving from theory to practice introduces noise and uncertainty at every layer. The backend you choose determines which effects are captured.

Backend	What it captures	Typical use
Statevector simulator	Ideal amplitudes, no sampling	Derivation and debugging
Shot simulator	Sampling noise only	Estimation experiments
Noisy simulator	Sampling & modeled hardware noise	Pre-hardware testing
Real hardware	Physical noise, drift & queue effects	Final validation

Key idea: The deterministic unitary model still defines the target behavior. Hardware deviations are quantified relative to that target.

Error Correction for Quantum Computing

"But what can you do with this toolkit?"

- Grover's Algorithm (*quadratic speedup for unstructured search*)
- Shor's Algorithm (*exponential speedup for integer factoring*)

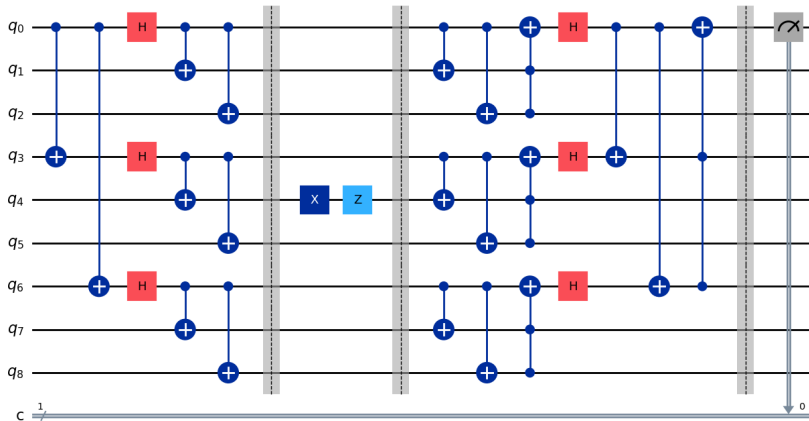
Classical computers protect data with **redundancy**:

- A **parity bit** detects single-bit flips by checking whether the number of 1s is even or odd.
- **Repetition codes** go further: encode $0 \rightarrow 000$ and $1 \rightarrow 111$, then correct by majority vote.

Quantum error correction faces two new obstacles:

- **No cloning** — quantum states cannot be copied, so naive repetition is forbidden.
- **Measurement destroys superposition** — you cannot just “look at” the qubits to find the error.

Shor's Repetition Code (9-Qubit)



Does this remind anyone of a classical repetition code? ($0 \rightarrow 000$ and $1 \rightarrow 111$)

$\Rightarrow$ This circuit can correct one bit flip!

Limitations for this workshop

Grover and Shor show real algorithmic ideas, but they are not plug-in tools for every business problem. They solve very specific issues.

- Error correction is required for large, deep circuits.
- Constant factors matter before asymptotics matter.
- Classical heuristics are strong baselines for many structured tasks.

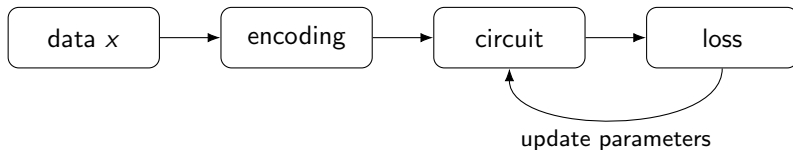
Connection to the workshop

These algorithms justify learning the circuit model: amplitudes, interference, measurement, and bitstring readout.

The practical workshop path then moves to shorter **parametrised** circuits and classical verification.

Notebook: `notebooks/story/03_quantum_states_circuits_algorithms.ipynb`.

Hybrid quantum machine learning



The optimiser is classical. The circuit is the trainable model component.

Lab file: 04.hybrid_quantum_classical_ml.ipynb

Section: Hybrid Architecture Overview

Hybrid model in one equation

A parametrised quantum circuit prepares

$$|\psi_\theta(x)\rangle = U_L(\theta_L)V_L(x)\cdots U_1(\theta_1)V_1(x)|0\cdots 0\rangle.$$

Prediction often uses an expectation value:

$$\hat{y}_\theta(x) = \langle \psi_\theta(x) | O | \psi_\theta(x) \rangle.$$

Lab file: 04_hybrid_quantum_classical_ml.ipynb

Section: Quantum Neural Network (QNN) Classifier

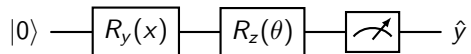
Outer loop remains classical

The training objective is classical:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_{\theta}(x_i), y_i).$$

The quantum subroutine supplies model evaluations. Optimisation, batching, validation, and early stopping are classical.

A one-qubit trainable block



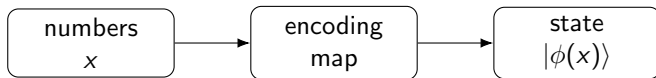
Input x rotates the state; parameter θ is trained.

Encoding: the first modelling choice

A circuit starts in $|0 \cdots 0\rangle$.

Classical data must first become a quantum state:

$$x \mapsto |\phi(x)\rangle.$$



Different encodings can make the same dataset easy or hard for the circuit.

Angle encoding: one number

A feature becomes a gate angle:

$$R_y(x_j) = \begin{pmatrix} \cos(x_j/2) & -\sin(x_j/2) \\ \sin(x_j/2) & \cos(x_j/2) \end{pmatrix}.$$

Exact example:

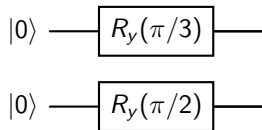
$$R_y(\pi/3)|0\rangle = \frac{\sqrt{3}}{2}|0\rangle + \frac{1}{2}|1\rangle.$$

Measuring gives $P(0) = 3/4$ and $P(1) = 1/4$.

Angle encoding: two features

For $x = (\pi/3, \pi/2)$, use one rotation per qubit:

$$|\phi(x)\rangle = R_y(\pi/3)|0\rangle \otimes R_y(\pi/2)|0\rangle.$$

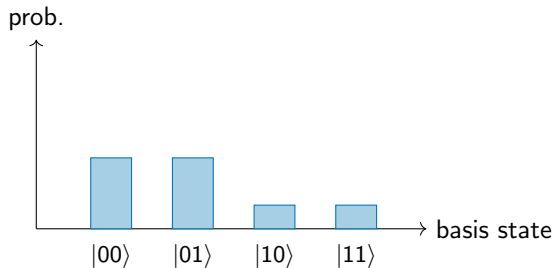


The feature scale matters: changing units changes the rotation angles.

Angle encoding: probabilities

For $x = (\pi/3, \pi/2)$:

$$|\phi(x)\rangle = \frac{\sqrt{6}}{4}|00\rangle + \frac{\sqrt{6}}{4}|01\rangle + \frac{\sqrt{2}}{4}|10\rangle + \frac{\sqrt{2}}{4}|11\rangle.$$



Amplitude encoding

Amplitude encoding stores a normalised vector directly in amplitudes:

$$|x\rangle = \sum_{j=0}^{2^n-1} x_j |j\rangle.$$

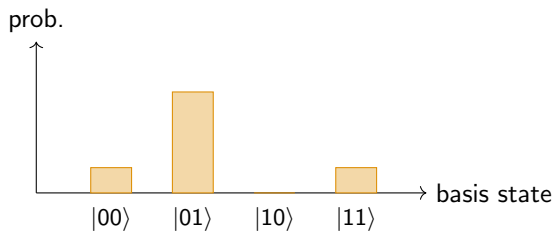
Example:

$$(1, 2, 0, 1) \mapsto \frac{1}{\sqrt{6}}|00\rangle + \frac{2}{\sqrt{6}}|01\rangle + 0|10\rangle + \frac{1}{\sqrt{6}}|11\rangle.$$

Lab file: 04.hybrid_quantum_classical_ml.ipynb

Section: Amplitude encoding: one vector

Amplitude encoding: what changes



It uses only two qubits for four numbers, but preparing a generic amplitude vector can require many gates.

Basis and Hamiltonian encodings

Basis encoding maps discrete patterns directly:

$$z = 101 \mapsto |101\rangle.$$

Optimisation problems often use Hamiltonian or cost observables:

$$H_C|z\rangle = E(z)|z\rangle.$$

Lab file: 04.hybrid_quantum_classical_ml.ipynb

Section: Prerequisite mini-lab · Data encoding for hybrid circuits

Variational quantum algorithms

A common variational form minimises

$$C(\theta) = \langle 0 | U^\dagger(\theta) H U(\theta) | 0 \rangle.$$

Examples: variational quantum eigensolver (VQE), quantum approximate optimisation algorithm (QAOA), quantum neural networks.

Lab file: 04_hybrid_quantum_classical_ml.ipynb

Section: Quantum Neural Network (QNN) Classifier

Hybrid finance example

Portfolio selection can be written as a cost over asset bitstrings:

$$E(z) = 0.6 \text{ risk}(z) - \text{return}(z) + 2 \left(\sum_i z_i - 2 \right)^2.$$

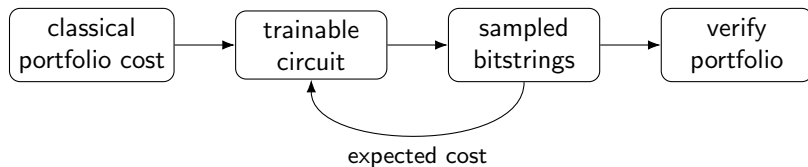
bitstring	selected assets	interpretation
110	A,B	two assets, feasible
101	A,C	two assets, feasible
111	A,B,C	too many, penalised

A trainable circuit changes the sampling probabilities over these strings.

Lab file: 04.hybrid_quantum_classical_ml.ipynb

Section: Part B · Financial example: portfolio selection

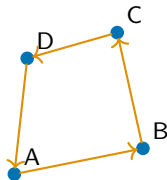
Hybrid finance workflow



This mirrors the standard QAOA-style idea in a small simulator example: optimise parameters classically, measure candidate strings quantumly.

Hybrid logistics example

Two measured qubits can choose between four candidate routes:



Classical code computes route lengths; the circuit is trained to sample shorter routes more often.

Lab file: 04.hybrid.quantum.classical.ml.ipynb

Section: Part C · Logistics example: route choice

Parameter-shift rule

For a gate generated by a Pauli observable,

$$U(\theta) = e^{-i\theta P/2},$$

the derivative of an expectation can be evaluated as

$$\partial_{\theta} \langle O \rangle_{\theta} = \frac{1}{2} (\langle O \rangle_{\theta+\pi/2} - \langle O \rangle_{\theta-\pi/2}).$$

Lab file: 04.hybrid_quantum_classical_ml.ipynb

Section: Training loop

Sampling cost

On hardware, an expectation is estimated from shots.

If a Pauli measurement has variance σ^2 , then

$$\text{Var}(\hat{\mu}) = \frac{\sigma^2}{S}$$

for S shots.

More shots reduce statistical error but increase wall-clock cost.

Barren plateaus

For some random deep ansätze, gradient variance can shrink exponentially with qubit count:

$$\text{Var} [\partial_{\theta_j} C(\theta)] \sim O(b^{-n}), \quad b > 1.$$

The optimiser then sees almost no signal.

Mitigation ideas

Common practical responses:

- use shallow, problem-informed ansätze;
- initialise near identity or train layerwise;
- choose local cost functions when possible;
- compare convergence and generalisation against classical baselines.

What we hope to gain

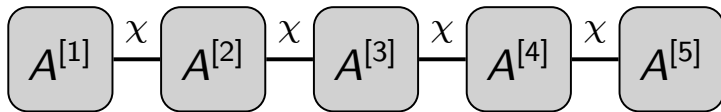
Hybrid quantum ML augments a classical workflow with a structured probabilistic circuit. The hope is not simply “more accuracy”. A quantum circuit may provide a useful inductive bias: correlations through entangling gates, compact feature maps, or sampling distributions that are hard for a simple classical model to imitate.

The claim is empirical: faster convergence, less overfitting, or better samples must be demonstrated against strong classical baselines.

Matrix Product State (MPS)

A quantum state with 2^N amplitudes is rewritten as a chain of small tensors:

$$|\psi\rangle \approx \sum_{\alpha_1, \dots, \alpha_{N-1}} A_{i_1 \alpha_1}^{[1]} A_{\alpha_1 i_2 \alpha_2}^{[2]} \cdots A_{\alpha_{N-1} i_N}^{[M]} |i_1 \cdots i_N\rangle.$$



The **bond dimension** χ controls the cost and accuracy of the representation:

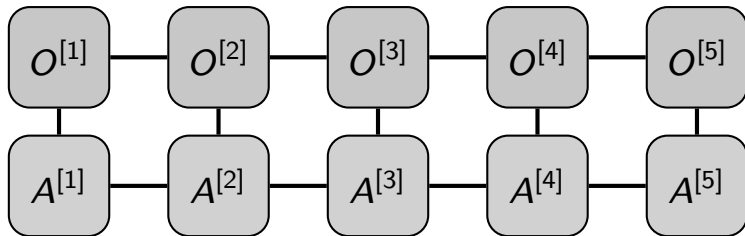
- Small χ : efficient but approximate; valid when entanglement is low
- $\chi = 2^{N/2}$: exact statevector, no compression

Each two-qubit gate can at most double χ , so circuits with limited entanglement growth remain simulable well beyond statevector limits.

Matrix Product Operator (MPO)

An MPO represents operators (Hamiltonians, noise channels) as a tensor chain, with both a ket and bra index per site.

MPO applied to an MPS



- Contraction grows the bond dimension; **truncation** keeps it tractable
- Analogous to a sparse matrix–vector product followed by compression

Quantum-inspired tensor networks

MPS/MPO structures are purely classical tools: bound the bond dimension, compress high-dimensional data, control the accuracy–compression tradeoff.

The notebook applies tensor-train decomposition to a language model weight matrix and measures the accuracy–compression tradeoff across ranks.

[notebooks/story/05_tensor_networks_quantum_inspired_llm.ipynb](#)

Why quantum-inspired methods appear

Near-term quantum hardware is noisy and limited in qubit count. In the meantime, the tensor structures developed for quantum simulation transfer directly to classical computation.

- Tensor networks compress high-dimensional arrays by bounding rank
- The gain is memory and parameter efficiency, not quantum speedup

From many-body states to tensors

A quantum state of d subsystems has 2^d amplitudes packed into a coefficient tensor:

$$|\psi\rangle = \sum_{i_1, \dots, i_d} T_{i_1, \dots, i_d} |i_1, \dots, i_d\rangle.$$

Storing T exactly is exponentially expensive.

Tensor networks factor it into a chain of small tensors, each coupled only to its neighbors, making the representation tractable when entanglement is low.

Tensor-train factorisation

A tensor $W \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ is factored into a chain of small matrices:

$$W_{i_1, \dots, i_d} = G_1[i_1] G_2[i_2] \cdots G_d[i_d],$$

where each core $G_k[i_k]$ is an $r_{k-1} \times r_k$ matrix with $r_0 = r_d = 1$.

The **ranks** r_k play the same role as bond dimension in MPS: larger ranks give a more accurate approximation at higher memory cost. Storage drops from $\prod_k n_k$ to $O(d n r^2)$.

Ranks control expressivity

- Dense tensor: $\prod_{k=1}^d n_k$ parameters, exponential in d
- Tensor-train: $\sum_{k=1}^d r_{k-1} n_k r_k$ parameters, linear in d

Smaller ranks compress more aggressively but limit which tensors can be represented exactly. Rank is the accuracy–efficiency knob.

Matrix compression pattern

A weight matrix $W \in \mathbb{R}^{m \times n}$ is reshaped into a higher-order tensor and factored into tensor-train cores:

$$W \rightarrow \mathcal{W}_{i_1, \dots, i_d} \rightarrow G_1 G_2 \cdots G_d.$$

- The dense matrix is never materialised at inference time
- Cores are contracted on the fly, reducing memory and compute
- Rank controls the compression ratio and approximation error

CompactifAI: The LLM Perspective

LLaMA-2 7B: 7 billion parameters, 27 GB to store. And this is the small version.

An LLM is a stack of weight matrices. Each matrix connects one layer to the next. To run the model, every matrix must fit in GPU memory, or be paged in and out, which is slow.

Classical compression options:

- Quantization: lower numerical precision
- Pruning: remove small weights
- Distillation: train a smaller copy

What they all miss:

- None reduce the number of parameters
- None ask which *correlations* actually carry information

The Key Insight: Weight Matrices Are Low-Rank

A 4096×4096 weight matrix has over 16 million entries. But in a trained LLM, the rows are not all independent.

The matrix is **low-rank**: most of its information lives in a small number of directions. The remaining dimensions are near-zero redundancy.

This means you can approximate W with far fewer numbers, if you know which directions to keep.

Empirical check: CompactifAI removes 70% of parameters and loses only 2–3% accuracy. There is a significant amount of redundancy!

Compressing a Matrix into a Chain

Instead of storing the full W , decompose it into a chain of smaller matrices linked together, a Matrix Product Operator (MPO).

Concretely for a 4096×4096 matrix:

- 1 Reshape W into a higher-order tensor
- 2 Apply Singular Value Decomposition (SVD) repeatedly along each axis
- 3 At each step, keep only the top χ singular values; discard the rest
- 4 Result: a chain of small tensors that multiply back to an approximation of W

The chain, when expanded, is close to the original W , but takes far less memory. The discarded singular values correspond to the near-zero redundant directions.

The analogy: quantum states and neural weights

Both systems exploit hidden structure to compress a massively redundant space.

Quantum state	LLM weight matrix
Exponentially large Hilbert space	Full $m \times n$ matrix
Local entanglement structure	Low effective rank
MPS/MPO with bond dimension χ	Compressed chain of tensors

- The connection is mathematical, not physical: MPS and tensor trains are the same decomposition in different communities
- Entanglement intuition motivates where low-rank approximations are valid
- No quantum hardware involved; all computation runs via classical SVD

Results: Size and Speed

Model	Memory	Params
Original (f32)	27.1 GB	7.0B
4-bit quantization	3.4 GB	7.0B
CompactifAI 88%	4.1 GB	2.1B
CompactifAI 93%	2.1 GB	2.1B

Speed on 8× A10g GPUs:

- Training (healing pass): 2× faster: fewer params to move between GPUs
- Inference (text generation): 25% faster: less data loaded per token
- 4-bit quantization: 13% *slower*: A10g has no native 4-bit units, so weights must be dequantized to f32 before each operation

Results: Accuracy at 93% Compression

Benchmark	Original	4-bit quant	CompactifAI 93%
MMLU (language understanding)	46.4	45.5	44.2
HellaSwag (commonsense)	80.6	79.3	76.5
BoolQ (reading comprehension)	79.8	78.2	76.8
GSM8K (math)	23.1	22.4	17.7

Most tasks: 2–3 point drop. Math drops more: reasoning is more sensitive to the correlation structure being truncated.

70% fewer parameters, 2–3% accuracy loss: LLMs are massively overparameterized.

Limitations

- Fine-tuning after compression is required; layer-by-layer MPO ignores cross-layer interactions
- Bond dimension χ must be tuned per model family; no universal value
- Low rank can remove task-relevant information
- Contraction overhead can offset memory savings at inference time
- Only tested on LLaMA-2 decoder-only transformers
- “Quantum-inspired” is partially a marketing label: MPO decomposition is mathematically equivalent to tensor-train decomposition, a classical tool

Three meanings of quantum AI

Quantum AI is not one method.

- Physics-informed: put known equations into the learning objective.
- Physics-enhanced: use quantum circuits as structured probabilistic models.
- Physics-inspired: use tensor structure classically when hardware is not ready.

The same discipline applies throughout: encode the problem, test a baseline, verify the output.

References

-  M. Raissi, P. Perdikaris, G.E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 2019.
-  G.E. Karniadakis et al. Physics-informed machine learning. *Nature Reviews Physics*, 2021.
-  L.K. Grover. A fast quantum mechanical algorithm for database search. STOC, 1996.
-  P.W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. FOCS, 1994.
-  L. Cerezo et al. Variational quantum algorithms. *Nature Reviews Physics*, 2021.
-  J.R. McClean, S. Boixo, V.N. Smelyanskiy, R. Babbush, H. Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 2018.
-  I.V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 2011.
-  R. Orus. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics*, 2014.

Please take a moment and tell us about your experience



SDS2026
DATA. IMPACT. VALUE.

Thank you!

